

מטלה בלמידת מכונה – ניתוח טקסט (NLP)

סיווג הודעות SMS כ-Spam / Ham עם Naive Bayes (מומש מאפס)

סוג מטלה: ניתוח טקסט (NLP) **סוג למידה:** Classification (סיווג בינארי) **אלגוריתם:** Naive Bayes — מומש עצמאי (fit + predict), ללא שימוש בספרייה מוכנה לאלגוריתם עצמו
Dataset: SMS Spam Collection (5,574 הודעות SMS מתויגות כ-ham/spam) **קישור ל-dataset ב-Kaggle:** <https://www.kaggle.com/datasets/team-ai/spam-text-messageclassification>

פרטי הסטודנטים

(השלימו כאן את שמכם: שם פרטי + אות ראשונה של שם משפחה + 4 ספרות אחרונות ת.ז., לשני בני הזוג)

- סטודנט/ית 1: _____
- סטודנט/ית 2: _____

פרומפטים ב-AI Chatbot ועזרים נוספים

במסגרת הכנת מטלה זו נעזרנו ב-Claude (Anthropic) לצורך בניית ה-pipeline. להלן תיאור השימוש:

- בקשה 1:** הסבר כללי על מבנה המטלה ודרישותיה, על סמך מסמך ההנחיות של הקורס.
- בקשה 2:** המלצה על אלגוריתם וסוג בעיה מתאימים לניתוח טקסט (הומלץ Naive Bayes + סיווג בינארי).
- בקשה 3:** בקשה לבצע את המטלה בפועל — טעינת דאטה, feature engineering, מימוש Naive Bayes מאפס, אימון, הרצת hyperparameter search, הערכת ביצועים על ה-test, והפקת מסמך הסבר.

קישורים נוספים בהם נעזרנו:

- SMS Spam Collection dataset (מקור: UCI Machine Learning Repository / Kaggle mirror)

מטרת השימוש: האצת תהליך הפיתוח וקבלת מבנה מסודר לעבודה, תוך הבנה מלאה של כל שלב (מוסבר לאורך המחברת ובסרטון).

הערה טכנית: גישה ישירה ל-Kaggle לא הייתה זמינה בסביבת הפיתוח, ולכן הנתונים נטענו ממראה (mirror) ציבורי זהה בתוכנו של אותו dataset ב-GitHub.

חלק 1 - הקדמה: הסבר על הבעיה וה-Dataset

הבעיה: סיווג הודעות SMS כ"ספאם" (הודעת פרסומת/הונאה לא רצויה) או "ham" (הודעה לגיטימית), על בסיס תוכן הטקסט של ההודעה בלבד. זוהי בעיית סיווג בינארי קלאסית בתחום ניתוח הטקסט (NLP).

ה-Dataset: SMS Spam Collection — אוסף של 5,574 הודעות SMS באנגלית, שכל אחת מתויגת כ- ham (לגיטימית) או spam. הדאטהסט מאוזן בצורה לא סימטרית: כ-87% מההודעות הן ham וכ-13% הן spam, מה שהופך את המחלקה "spam" למחלקה המרכזית שעליה נמדוד F1 בהמשך.

הערה לגבי חלוקת train/test: הגרסה שהורדנו אינה כוללת חלוקה מובנית מראש ל- train/test (בניגוד לחלק מהדאטהסטים ב-Kaggle). לכן ביצענו **חלוקה יחידה, בתחילת התהליך בלבד** (train / 20% test 80%, עם stratify לפי התיג), ולאחר מכן **לא נגענו שוב ב-test** עד לשלב ההערכה הסופי בחלק 5 — בדיוק כפי שנדרש במטלה עבור דאטהסט המגיע כבר מחולק.

```
In [1]: import pandas as pd
import numpy as np
import re
from sklearn.model_selection import train_test_split
from sklearn.metrics import f1_score, accuracy_score, precision_score, recall_score

pd.set_option("display.max_colwidth", 80)

# טעינת הדאטה
# יכולים להיות שמות עמודות שונים בין גרסאות Kaggle-שימו לב: לקובץ שמורידים מ-
# לכן מזהים אוטומטית - (בגרסה אחרת Category/Message, בגרסה אחת v1/v2 למשל)
df = pd.read_csv("spam.csv", encoding="latin-1")

if {"v1", "v2"}.issubset(df.columns):
    df = df[["v1", "v2"]]
elif {"Category", "Message"}.issubset(df.columns):
    df = df[["Category", "Message"]]
else:
    raise KeyError(
        f"{list(df.columns)}: לא זוהו שמות עמודות מוכרים בקובץ. העמודות שנמצאו"
    )

df.columns = ["label", "text"]
df["label_num"] = (df["label"] == "spam").astype(int) # 1 = spam, 0 = ham

print("סה"כ הודעות בדאטהסט:", df.shape[0])
print(df["label"].value_counts())
df.head()
```

סה"כ הודעות בדאטהסט: 5572
label
ham 4825
spam 747
Name: count, dtype: int64

```
Out[1]:
```

	label	text	label_num
0	ham	Go until jurong point, crazy.. Available only in bugis n great world la e bu...	0
1	ham	Ok lar... Joking wif u oni...	0
2	spam	Free entry in 2 a wkly comp to win FA Cup final tkts 21st May 2005. Text FA ...	1
3	ham	U dun say so early hor... U c already then say...	0
4	ham	Nah I don't think he goes to usf, he lives around here though	0

```
In [2]: # (הדאטהסט המקורי אינו מגיע מחולק train/test-חלוקה חד-פעמית ל  
# עד חלק 5 בלבד test-מדרג זה ואילך אין נגיעה ב  
train_df, test_df = train_test_split(  
    df, test_size=0.2, stratify=df["label_num"], random_state=42  
)  
train_df = train_df.reset_index(drop=True)  
test_df = test_df.reset_index(drop=True)  
  
print("Train:", train_df.shape, " Test:", test_df.shape)  
print("\nהתפלגות תוויות בTrain:")  
print(train_df["label"].value_counts(normalize=True))  
print("\nהתפלגות תוויות בTest:")  
print(test_df["label"].value_counts(normalize=True))
```

Train: (4457, 3) Test: (1115, 3)

התפלגות תוויות ב Train:

label
ham 0.865829
spam 0.134171
Name: proportion, dtype: float64

התפלגות תוויות ב Test:

label
ham 0.866368
spam 0.133632
Name: proportion, dtype: float64

5 השורות הראשונות של ה-Train set:

```
In [3]: train_df.head()
```

Out[3]:	label	text	label_num
0	ham	He will, you guys close?	0
1	ham	CAN I PLEASE COME UP NOW IMIN TOWN.DONTMATTER IF URGOIN OUTL8R,JUST REALLYNE...	0
2	ham	Ok k..sry i knw 2 siva..tats y i askd..	0
3	ham	I'll see, but prolly yeah	0
4	ham	I'll see if I can swing by in a bit, got some things to take care of here firsg	0

5 השורות הראשונות של ה-Test set:

In [4]: `test_df.head()`

Out[4]:	label	text	label_num
0	ham	No need to buy lunch for me.. I eat maggi mee..	0
1	ham	Ok im not sure what time i finish tomorrow but i wanna spend the evening wit...	0
2	ham	Waiting in e car 4 my mum lor. U leh? Reach home already?	0
3	spam	You have won ?1,000 cash or a ?2,000 prize! To claim, call09050000327	1
4	ham	If you r @ home then come down within 5 min	0

חלק 2 - Feature Engineering

כדי שהאלגוריתם יוכל לעבוד עם טקסט, יש להפוך אותו לייצוג מספרי. השלבים שביצענו:

1. **המרה לאותיות קטנות** — כדי ש-"Free" ו-"free" יחשבו לאותה מילה.
2. **הסרת סימני פיסוק ומספרים** — משאירים רק אותיות ורווחים.
3. **טוקניזציה** — פיצול הטקסט למילים בודדות (tokens).
4. **הסרת stop-words** — מילים נפוצות שאינן נושאות משמעות סיווגית (, is , the , and , וכו').
5. **ייצוג Bag-of-Words** — לכל הודעה, רשימת המילים (tokens) שנשארו לאחר הניקוי היא הבסיס לחישובי ה-Naive Bayes (ספירת שכיחות מילים לפי מחלקה).

מאחר ונדרשה מימוש עצמאי של האלגוריתם (חלק 3), גם שלב ה-vectorization (ספירת מילים) ממומש כאן ידנית, ולא באמצעות `CountVectorizer` המוכן של `scikit-learn`.

In [5]: `STOPWORDS = set("""a an the and or but if is are was were be been being to c
as at by from this that these those it its it's i you he she we they my your
me him them not no do does did so than then there here just so very can will
u ur 2 4""").split())`

```
def clean_and_tokenize(text):
    text = text.lower()
    text = re.sub(r"[^a-z\s]", " ", text) # השארת אותיות ורווחים בלבד
    tokens = text.split()
    tokens = [t for t in tokens if t not in STOPWORDS and len(t) > 1]
    return tokens

# הדגמה על 3 דוגמאות מה מ
print("=== דוגמאות-Train ===")
for i in range(3):
    raw = train_df.loc[i, "text"]
    print("RAW      :", raw)
    print("TOKENS:", clean_and_tokenize(raw))
    print()
```

=== דוגמאות-Train ===

RAW : He will, you guys close?

TOKENS: ['guys', 'close']

RAW : CAN I PLEASE COME UP NOW IMIN TOWN.DONTMATTER IF URGON OUTL8R,JUST
REALLYNEED 2DOCD.PLEASE DONTPLEASE DONTIGNORE MYCALLS,U NO THECD ISV.IMPORTA
NT TOME 4 2MORO

TOKENS: ['please', 'come', 'up', 'now', 'imin', 'town', 'dontmatter', 'urgoi
n', 'outl', 'reallyneed', 'docd', 'please', 'dontplease', 'dontignore', 'myc
alls', 'thecd', 'isv', 'important', 'tome', 'moro']

RAW : Ok k..sry i knw 2 siva..tats y i askd..

TOKENS: ['ok', 'sry', 'knw', 'siva', 'tats', 'askd']

In [6]: # הדגמה על 3 דוגמאות מה מ

```
print("=== דוגמאות-Test ===")
for i in range(3):
    raw = test_df.loc[i, "text"]
    print("RAW      :", raw)
    print("TOKENS:", clean_and_tokenize(raw))
    print()
```

=== דוגמאות-Test ===

RAW : No need to buy lunch for me.. I eat maggi mee..

TOKENS: ['need', 'buy', 'lunch', 'eat', 'maggi', 'mee']

RAW : Ok im not sure what time i finish tomorrow but i wanna spend the eve
ning with you cos that would be vewy vewy lubly! Love me xxx

TOKENS: ['ok', 'im', 'sure', 'what', 'time', 'finish', 'tomorrow', 'wanna',
'spend', 'evening', 'cos', 'vewy', 'vewy', 'lubly', 'love', 'xxx']

RAW : Waiting in e car 4 my mum lor. U leh? Reach home already?

TOKENS: ['waiting', 'car', 'mum', 'lor', 'leh', 'reach', 'home', 'already']

```
In [7]: # (בנפרד train ו-test) על כל הדאטה feature engineering-הפעלת ה
train_tokens = [clean_and_tokenize(t) for t in train_df["text"]]
test_tokens = [clean_and_tokenize(t) for t in test_df["text"]]

print("מספר הודעות מעובדות ב-train:", len(train_tokens))
print("מספר הודעות מעובדות ב-test :", len(test_tokens))
```

ב מספר הודעות מעובדות ב-train: 4457
 ב מספר הודעות מעובדות ב-test : 1115

חלק 3 – מימוש אלגוריתם: Naive Bayes (מאפס)

העיקרון: Naive Bayes מבוסס על חוק בייס, בהנחה (ה"נאיבית") שהמילים בהודעה בלתי-תלויות זו בזו בהינתן המחלקה. עבור הודעה חדשה עם מילים $w_1, w_2, \dots, w_n$, אנו בוחרים את המחלקה c שממקסמת:

$$P(c \mid w_1, \dots, w_n) \propto P(c) \cdot \prod_{i=1}^n P(w_i \mid c)$$

כאשר:

- $P(c)$ — ההסתברות המקדמית (Prior) של המחלקה, נאמדת כשכיחות המחלקה ב-train.
- $P(w_i \mid c)$ — ההסתברות של המילה w_i בהינתן המחלקה c , נאמדת מתוך ספירת המילים ב-train, עם **Laplace smoothing** (היפרפרמטר α) כדי להימנע מהסתברות אפס עבור מילים שלא נראו במחלקה מסוימת:

$$P(w \mid c) = \frac{\text{count}(w, c) + \alpha}{\sum_{w'} \text{count}(w', c) + \alpha}$$

בפועל עובדים עם **log-probabilities** (סכום לוגריתמים במקום מכפלה) כדי למנוע underflow מספרי.

המחלקה שלמטה מממשת שתי פונקציות עיקריות, כנדרש:

- **fit** — לומדת מה-train את ה-priors וה-word counts לכל מחלקה.
 - **predict** — מסווגת הודעות חדשות לפי כלל ה-MAP (Maximum A Posteriori).
- ההיפרפרמטר (Laplace smoothing) α ניתן לכיוונון, כפי שנלמד בכיתה.

```
In [8]: class NaiveBayesTextClassifier:
        """
        לסיווג טקסט Multinomial Naive Bayes של (from scratch) מימוש עצמאי.
        alpha: היפרפרמטר Laplace smoothing.
        """
        def __init__(self, alpha: float = 1.0):
            self.alpha = alpha

        def fit(self, tokenized_docs, labels):
            self.classes_ = sorted(set(labels))
            n_docs = len(labels)
```

```

self.class_priors_ = {
    c: sum(1 for l in labels if l == c) / n_docs
    for c in self.classes_
}

self.word_counts_ = {c: {} for c in self.classes_}
self.total_words_ = {c: 0 for c in self.classes_}
vocab = set()

for tokens, label in zip(tokenized_docs, labels):
    for w in tokens:
        vocab.add(w)
        self.word_counts_[label][w] = self.word_counts_[label].get(w, 0)
        self.total_words_[label] += 1

self.vocab_ = vocab
self.vocab_size_ = len(vocab)
return self

def _log_likelihood(self, word, c):
    count = self.word_counts_[c].get(word, 0)
    return np.log(
        (count + self.alpha) / (self.total_words_[c] + self.alpha * self.vocab_size_)
    )

def predict_one(self, tokens):
    scores = {}
    for c in self.classes_:
        score = np.log(self.class_priors_[c])
        for w in tokens:
            if w in self.vocab_:
                # מילה לא מוכרת כלל - מדלגים
                score += self._log_likelihood(w, c)
        scores[c] = score
    return max(scores, key=scores.get)

def predict(self, tokenized_docs):
    return [self.predict_one(t) for t in tokenized_docs]

print("NaiveBayesTextClassifier מוכנה (fit + predict).")

```

NaiveBayesTextClassifier מוכנה (fit + predict). המחלקה

חלק 4 - אימון: הרצת ה-Flow עם פרמטרים שונים

לפני האימון הסופי, בדקנו כמה ערכים של ההיפרפרמטר (Laplace smoothing) `alpha`, על-מנת לבחור את הקומבינציה המוצלחת ביותר. לצורך זה **פיצלנו את ה-train בלבד** (לא נגענו ב-test) ל-train-פנימי ו-validation, מדדנו F1 על ה-validation לכל ערך `alpha`, ובחרנו את המנצח.

לאחר מכן, כנדרש, **אימנו מחדש על כל ה-trainset** (ללא הפיצול הפנימי) עם ה-`alpha` הנבחר.

```
In [9]: # (test-בלבד, לא נוגע ב hyperparameter search ל-train-פיצול פנימי של ה
tr_tok, val_tok, tr_lab, val_lab = train_test_split(
    train_tokens, list(train_df["label_num"]), test_size=0.2,
    stratify=train_df["label_num"], random_state=42
)

alpha_grid = [0.1, 0.5, 1.0, 2.0]
search_results = []

for alpha in alpha_grid:
    model = NaiveBayesTextClassifier(alpha=alpha).fit(tr_tok, tr_lab)
    val_preds = model.predict(val_tok)
    f1 = f1_score(val_lab, val_preds, pos_label=1)
    search_results.append((alpha, f1))
    print(f"alpha={alpha:>4} -> F1(validation) = {f1:.4f}")

best_alpha = max(search_results, key=lambda x: x[1])[0]
print(f"\n>>> נבחר alpha = {best_alpha} (F1 מקסימלי על ה validation)")

alpha= 0.1 -> F1(validation) = 0.9160
alpha= 0.5 -> F1(validation) = 0.9198
alpha= 1.0 -> F1(validation) = 0.9231
alpha= 2.0 -> F1(validation) = 0.9258

>>> alpha = 2.0 (F1 מקסימלי על ה validation)
```

```
In [10]: # הנבחר alpha-עם ה trainset-אימון סופי על כל ה
final_model = NaiveBayesTextClassifier(alpha=best_alpha).fit(
    train_tokens, list(train_df["label_num"])
)
print("האימון הסופי הושלם.")
print("שנלמד (vocabulary) גודל אוצר המילים:", final_model.vocab_size_)
print("Prior(ham) =", round(final_model.class_priors_[0], 4),
      " Prior(spam) =", round(final_model.class_priors_[1], 4))
```

האימון הסופי הושלם.
שנלמד: 6768 (vocabulary) גודל אוצר המילים
Prior(ham) = 0.8658 Prior(spam) = 0.1342

חלק 5 - חיזוי ושערוך איכות המודל על ה-Test Set

כעת, ורק כעת, אנו משתמשים ב-test set (בו לא נגענו עד כה) כדי להריץ predict ולמדוד את איכות המודל.

מאחר ומדובר בבעיית סיווג בינארי עם מחלקה מרכזית אחת (spam), מדד האיכות הנדרש הוא F1 על המחלקה המרכזית (spam) בלבד.

```
In [11]: test_preds = final_model.predict(test_tokens)

print("test-תחזיות ראשונות על ה 5:")
for i in range(5):
    true_label = "spam" if list(test_df['label_num'])[i] == 1 else "ham"
    pred_label = "spam" if test_preds[i] == 1 else "ham"
    text_preview = test_df.loc[i, "text"][:60]
    print(f"[{i}] אמת={true_label:5s} חזוי={pred_label:5s} | \"{text_preview}
```

```
5 test-תחזיות ראשונות על ה:
[0] אמת=ham      חזוי=ham      | "No need to buy lunch for me.. I eat maggi me
e....."
[1] אמת=ham      חזוי=ham      | "Ok im not sure what time i finish tomorrow but
i wanna spend..."
[2] אמת=ham      חזוי=ham      | "Waiting in e car 4 my mum lor. U leh? Reach hom
e already?..."
[3] אמת=spam     חזוי=spam     | "You have won ?1,000 cash or a ?2,000 prize! To
claim, call09..."
[4] אמת=ham      חזוי=ham      | "If you r @ home then come down within 5 min..."
```

```
In [12]: f1 = f1_score(test_df["label_num"], test_preds, pos_label=1)
acc = accuracy_score(test_df["label_num"], test_preds)
prec = precision_score(test_df["label_num"], test_preds, pos_label=1)
rec = recall_score(test_df["label_num"], test_preds, pos_label=1)
cm = confusion_matrix(test_df["label_num"], test_preds)

print(f"F1 (spam, המחלקה המרכזית) : {f1:.4f}")
print(f"Precision (spam) : {prec:.4f}")
print(f"Recall (spam) : {rec:.4f}")
print(f"Accuracy (כללי, למידע) : {acc:.4f}")
print()
print("Confusion Matrix:")
print("      pred=ham      pred=spam")
print(f"true=ham      {cm[0][0]:5d}      {cm[0][1]:5d}")
print(f"true=spam      {cm[1][0]:5d}      {cm[1][1]:5d}")
```

```
F1 (spam, המחלקה המרכזית) : 0.9273
Precision (spam) : 0.9571
Recall (spam) : 0.8993
Accuracy (כללי, למידע) : 0.9812
```

Confusion Matrix:

	pred=ham	pred=spam
true=ham	960	6
true=spam	15	134

המודל, שממומש כולו מאפס (ללא שימוש בספריית Naive Bayes מוכנה), מגיע ל-F1 גבוה על המחלקה המרכזית (spam) על ה-test set שלא נגענו בו לאורך התהליך. הביצועים הגבוהים תואמים לכך ש-Naive Bayes הוא אלגוריתם קלאסי ויעיל במיוחד לבעיות סיווג טקסט מהסוג הזה, בעיקר משום שהוא מנצל היטב את השכיחות המובחנת של מילות "דגל" (כמו, free, win, call, text, claim) בהודעות ספאם.